Search Algorithms

Matteo Fasiolo

What is Search?

Search algorithms find object(s) from **a candidate set** according to a **searching criterion**.

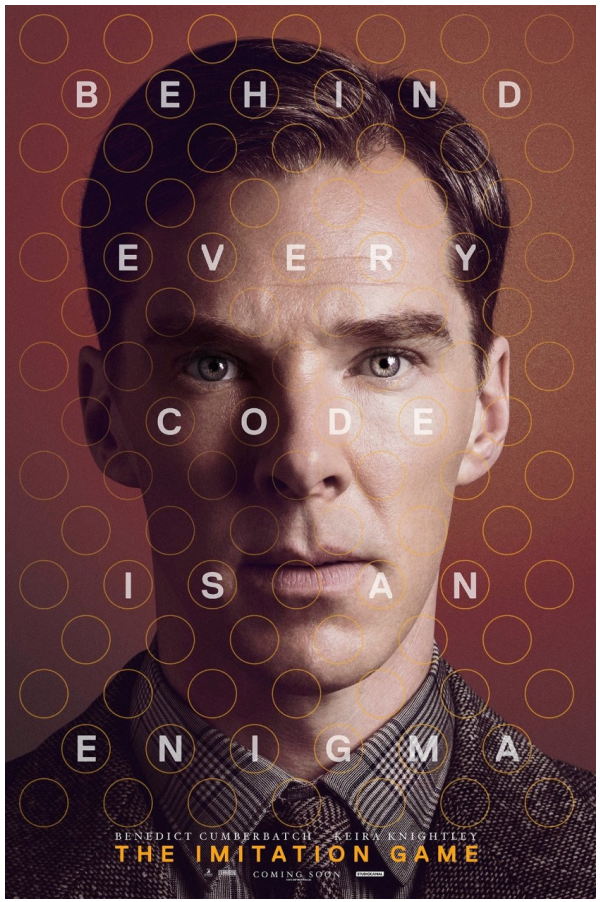
Many CS problems are search problems:

- Find the index of the maximum (smallest) element in an array.
- Find the index of a specific element in a sorted array.
- Find x that maximizes a function $f(x)$.
- Find the best move in a chess game.
- Find the key to a password encryption.

Cryptography

Cryptography is a classic search problem.

To decrypt messages, you need to find "keys" from a huge candidate set: The mathematician (Alan Turing) who tried to solve this problem invented modern computers.



Machine Learning

Many machine learning problems are search problems:

Find a model from a **model family** that minimizes/maximizes objective functions:

- A good weather prediction model should accurately predict temperatures with minimum error when comparing to the actual weather data.
- A good advertisement model should recommend user ads that leads to maximum click rates.
- Commonly used model families (such as neural networks) usually entail a huge search space.

Search Problem

More formally,

$$\{i^*\} = \operatorname*{argmin}_{i \in \mathcal{F}} f(i)$$

where $\mathcal{F}$ defines a search space and f defines the search criterion.

For example, finding the smallest element in a length n array:

$$\{i^*\} = \operatorname*{argmin}_{i \in \{1,\ldots,n\}} s_i,$$

where s_i is the i-th element in the array s.

Finding the index of the element closest to 4 in a length n array:

$$\{i^*\} = \operatorname*{argmin}_{i \in \{1,\ldots,n\}} |s_i - 4|,$$

When the search space is discrete, and is not large, we can enumerate the entire $\mathcal{F}$ to find the best fit(s).

- As you have done in `find_max_idx` function (Tut week 7).

However, what if the search space is infinite?

$$\{x^*\} = \underset{x \in \mathbb{R}}{\text{argmax}} f(x),$$

Do we search the entire real domain?

Local Search Algorithms

Local search algorithms is the name of **a set of search algorithms**.

1. Local search algorithm starts from a candidate solution a problem.
2. It searches for a better solution that is close to the current solution.
3. If it find a better solution, it searches for an even better solution in its neighbourhood.
4. Keep iterating until a certain stopping criterion is satisfied.

For local algorithms to work, the local problem must provide **enough information** to the original problem.

Hill Climbing

Consider the problem of finding the maximiser of a function:

$$\{x^*\} = \operatorname*{argmax}_{x \in \mathbb{R}} f(x),$$

How do we split this problem into meaningful local subproblems?

We can start at an arbitary location x_0 and solve the subproblem:

$$\{x_1^*\} = \underset{x \in [x_0 - \epsilon, x_0 + \epsilon]}{\text{argmax}} \quad f(x),$$

where $\epsilon > 0$ is a fixed value.

The next step, we solve

$$\{x_2^*\} = \underset{x \in [x_1^* - \epsilon, x_1^* + \epsilon]}{\text{argmax}} \quad f(x)$$

and so on...

- After finding the maximizer of the previous local problem, we restart search for the maximum centered around the previous maximizer.
- Hence the name "Hill Climbing".

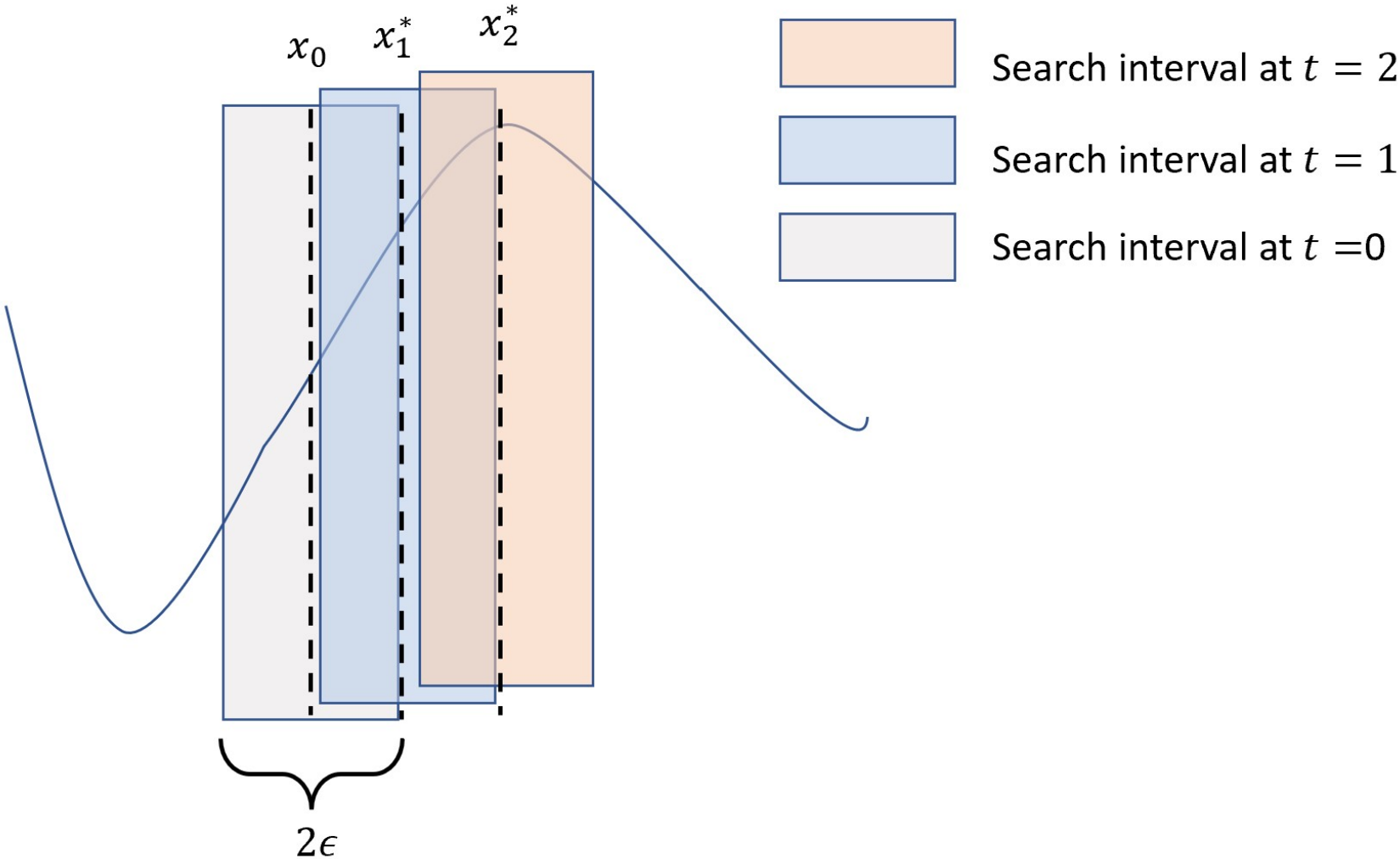
The algorithm stops when successive steps finds identical optimal solutions, i.e. $x^*_{t+1} = x^*_t$.

For each sub-problem,

$$\{x^*_{t+1}\} = \underset{x \in [x^*_t - \epsilon, x^*_t + \epsilon]}{\text{argmax}} \; f(x),$$

we can find an approximate solution via grid search:

1. Split $[x^*_t - \epsilon, x^*_t + \epsilon]$ into discrete "grid points".
   - e.g. $[-1, 1]$ can be discretized as $[-1, -0.9, \ldots, 0.9, 1]$.
2. Find grid point that gives the smallest function value.


x_0
x_1^*
x_2^*
2ϵ
Search interval at $t = 2$
Search interval at $t = 1$
Search interval at $t = 0$

The Subproblem

```
# This function returns xmax that maximizes f(x)
# between x-epsilon and x+epsilon
# complete the code yourself
subproblem <- function(xt, epsilon) {
  x <- xt - epsilon
  fmax <- -Inf
  xmax <- x
  while (x <= xt + epsilon) {
    val <- f(x)
    if (# Your code here) {
     # Your code here
    }
    x <- x + epsilon/100
  }
  return(xmax)
}
```

Running the algorithm

```
epsilon <- 0.5
x0 <- -4
xt <- x0
xt_1 <- 100

while (abs(xt - xt_1) >= 1e-5) {
  xt_1 <- xt
  # Your code here
  cat(sprintf("f(%.4f) = %.4f\n", xt, f(xt)))
}

cat(sprintf("Max is at %f with value %f\n", xt, f(xt)))
```

Output

```
f(-3.5000) = -0.3430
f(-3.0000) = -0.2658
f(-2.5000) = -0.1380
f(-2.0000) = 0.0361
f(-1.5000) = 0.2489
f(-1.0000) = 0.4895
f(-0.5000) = 0.7448
f(-0.0050) = 0.9975
f(0.4900) = 1.2351
f(0.9850) = 1.4427
f(1.4850) = 1.6080
f(1.9850) = 1.7168
f(2.4850) = 1.7597
f(2.5400) = 1.7602
f(2.5400) = 1.7602
Maximum is at 2.540000 with value 1.760173
```

Questions

1. Is this algorithm guaranteed to find the maximum of $f(x)$?
2. How will epsilon affect the performance of the algorithm?

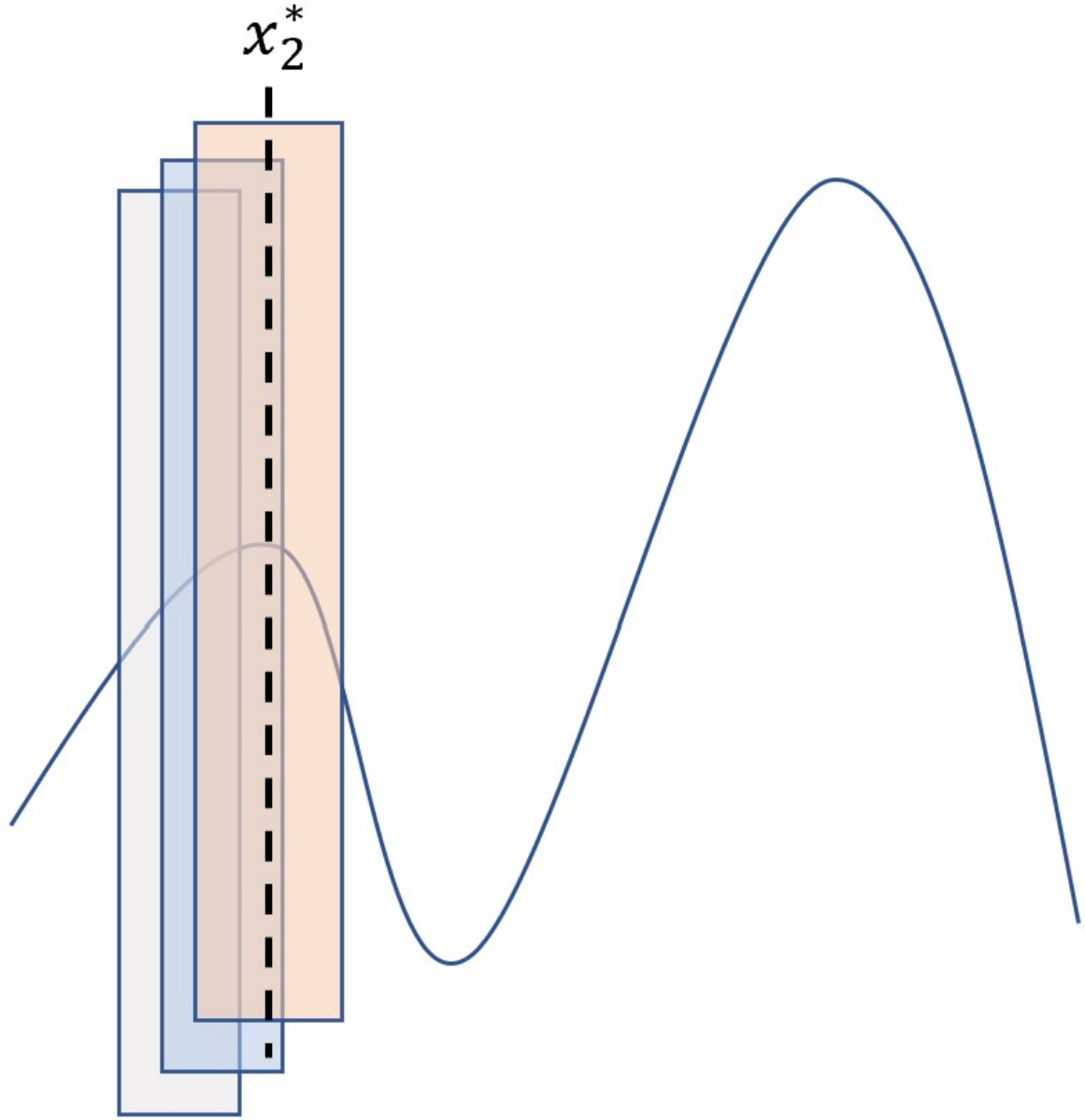
Local optimum

No, the algorithm is **not** guaranteed to find the maximum.

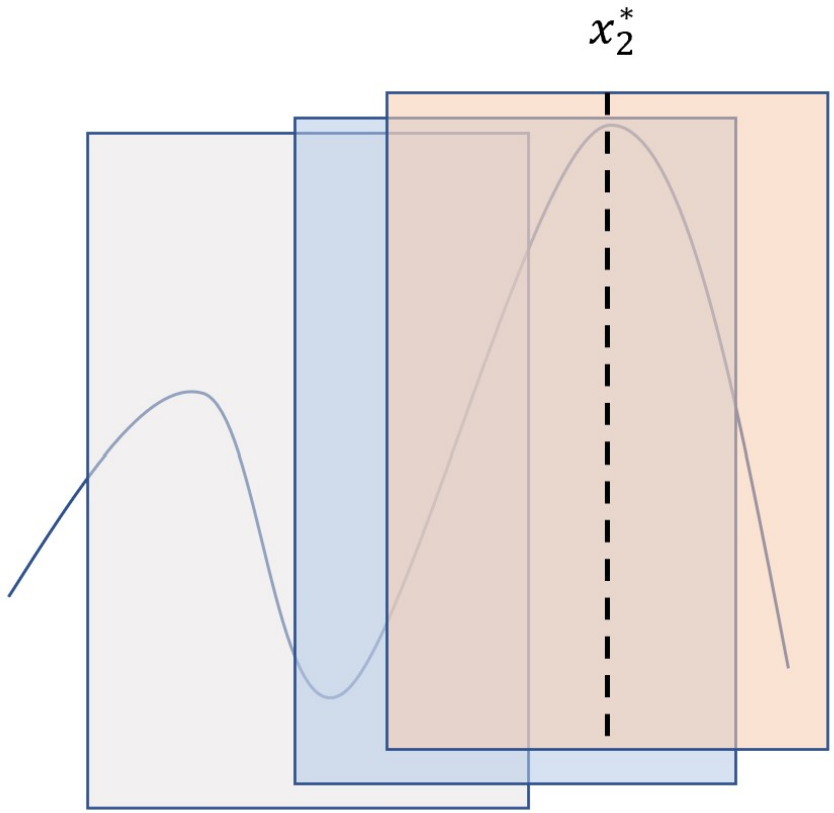
Solving subproblems successively may not lead to the global optimal solution.

The subproblems may not provide necessary information about **the global structure** of $f(x)$ to reach the global optimal solution.

It may only reach a solution that is **locally optimal**, that is the solution is optimal only within **the reach of our search space**.



Algorithm stuck at x_2^*!



With a bigger ϵ, algorithm is **not** stuck at local optimal!

However, bigger ϵ means solving the subproblem is more time-consuming (more grid points).

We have to make the trade off between a better solution and computational time.

TicTacToe

In some cases, the search space $\mathcal{F}$ is finite and well-defined, but the searching criteria is not.

Consider the game TicTacToe:

```
X O *
* O *
X X *
```

where * is an empty spot.

Players X and O try to connect their pieces as a straight line, which can be a column, a row, or a diagonal line.

```
X O *
* O O
X X X // X wins!
```

Given a specific game, what is the optimal move?

```
X O *
* * *
* * *
```

The “moves” are finite and well-defined (the empty spots on the board).

However, how do we define “the optimal move”?

Sure, a good move leads to winning the game, but how to specify optimality in an algorithmic manner?

A Greedy Algorithm

We can use a greedy approach.

Similarly to local search, the problem (the game) is divided into subproblems (turns).

We make the decision that seems optimal for the current subproblem.

That is, we make a decision considering the outcome of the current turn, not of the whole game.

Hence, at each turn we are greedy (and short-sighted).

Note, hill-climbing can be considered a local greedy search algorithm.

So, instead of aiming at "winning the game", we focus on the **next turn only**.

Let the game board be a 3 by 3 matrix.

Recall the game is trying to connect pieces into a straight line.

For a good move i, j,

- It should maximize the number of our own pieces on i-th row, j-th column and diagonal lines (if i, j is on the diagonal).
- It should avoid opponent pieces along these lines, which would prevent us from forming a line.

At step t, the optimal move is determined by

$$\{(i^*_{t+1}, j^*_{t+1})\} = \underset{(i,j)\in\mathcal{F}_t}{\text{argmax}}\, f(i,j),$$

where $\mathcal{F}_t$ is the set of feasible moves.

$f(i,j)$ scans i-th row, j-th column and diagonal lines (if i,j is on the diagonal or anti-diagonal).

The function value is the number of self-pieces on these lines **minus** the number of opponent-pieces.

Suppose M is the game board. Let $\sum_k = \sum_{k=1}^{3}$ and define

$$\begin{aligned} f(i,j) := &\sum_k \mathbb{1}(M_{i,k} = \text{self}) - \sum_k \mathbb{1}(M_{i,k} = \text{opp}) \\ &+ \sum_k \mathbb{1}(M_{k,j} = \text{self}) - \sum_k \mathbb{1}(M_{k,j} = \text{opp}) \\ &+ g(i,j) + h(i,j). \end{aligned}$$

Count along main diagonal:

$$g := \begin{cases} \sum_k \mathbb{1}(M_{k,k} = \text{self}) - \sum_k \mathbb{1}(M_{k,k} = \text{opp}), & i = j \\ 0, & i \neq j \end{cases}$$

Count along main anti-diagonal:

$$h := \begin{cases} \sum_k \mathbb{1}(M_{k,4-k} = \text{self}) - \sum_k \mathbb{1}(M_{k,4-k} = \text{opp}), & i + j = 4 \\ 0, & i + j \neq 4 \end{cases}$$

We want to create code that works like this:

```
game <- create_board()
print_board(game)
```

```
*  *  *
*  *  *
*  *  *
```

```
game <- play_move(game, 1, 1, "X")
print_board(game)
```

```
X  *  *
*  *  *
*  *  *
```

```
game <- ai_play(game, "O");
print_board(game)
```

```
X  *  *
*  *  O
*  *  *
```

We need `create_board` to create an empty board and `print_board` to print it.

A `play_move` function to play manually:

```
play_move <- function(board, i, j, player) {
  if ( is_playable(board, i, j) ) {
    board <- set_elem(board, i, j, player)
  }
  return( board )
}
```

when need to create `is_playable` and `set_elem`.

An `ai_play` function to play automatically:

```
ai_play <- function(board, player) {
  # TODO: It plays the next move for the "player",
  # plays the move (i,j) is the maximizer of f(i,j)
}
```

We need a function to evaluate $f(i, j)$:

```
evaluate_move <- function(board, i, j, player) {
  opponent <- if (player == "X") "O" else "X"
  f_ij <- 0
  f_ij <- f_ij + pieces_at_row(board, i, player)
  f_ij <- f_ij - pieces_at_row(board, i, opponent)
  f_ij <- f_ij + pieces_at_col(board, j, player)
  f_ij <- f_ij - pieces_at_col(board, j, opponent)
  if (i == j) {
    f_ij <- f_ij + pieces_at_diag(board, player)
    f_ij <- f_ij - pieces_at_diag(board, opponent)
  }
  if (i + j == 4) { # since i,j now run from 1-3
    f_ij <- f_ij + pieces_at_anti_diag(board, player)
    f_ij <- f_ij - pieces_at_anti_diag(board, opponent)
  }
  return( f_ij )
}
```

We need functions that count pieces:

```
pieces_at_row <- function(board, i, player) {
 # TODO: Count player's pieces at i-th row
}

pieces_at_col <- function(board, j, player) {
 # TODO: Count player's pieces at j-th column
}

pieces_at_diag <- function(board, player) {
 # TODO: Count player's pieces at diagonal line
}

pieces_at_anti_diag <- function(board, player) {
 # TODO: Count player's pieces at anti-diagonal line
}
```

Let it play!

```
game <- create_board()

game <- play_move(game, 1, 1, "X")
game <- play_move(game, 2, 1, "O")
game <- play_move(game, 1, 3, "X")
game <- play_move(game, 2, 3, "O")

print_board(game)
```

```
X   *   X
O   *   O
*   *   *
```

```
X  *  X
O  *  O
*  *  *
```

Now the AI plays

```
game <- ai_play(game, "X")
print_board(game)
```

```
X  X  X
O  *  O
*  *  *
```

It works!

However, is our strategy optimal?

A Suboptimal Move

Suppose board is:

```
X  O  *
X  X  O
*  *  *
```

The AI plays

```
game <- ai_play(game, "O")
print_board(game)
```

```
X  O  O
X  X  O
*  *  *
```

You can design a more complex subproblem anticipating your opponent's counter moves, but it would increase computational complexity too.

We have to compromise between computational time and optimality of our solution.

Conclusion

Search Problem finds objects(s) from a search space given a searching criterion.

Local/greedy algorithms:

- Hill Climbing
- TicTacToe

Both of them can find locally optimal solutions, but neither of them is guaranteed to find the global optimum.

Greedy algorithm is myopic, which means it only focuses on optimizing the subproblem, which may NOT lead to **the global optimal solution**.